

MCP/A2A Infrastructure & Agentic Routing

AICB-P2T2 · Ngày 26 · Chương 6: Tổng Hợp

VinUniversity · Phase 2 · Track 2 · Tuần 6

HÃY SUY NGHĨ...

"Agent gọi agent — infra của bạn có scale được cho multi-agent choreography không?"

Case study: Research system cần 4 specialized agents phối hợp — nếu không có chuẩn giao tiếp, mỗi agent nói "ngôn ngữ" riêng.

MCP + A2A = universal protocol cho AI agent ecosystem.

Giữ câu hỏi này trong đầu suốt buổi học hôm nay

Nội Dung Bài Học

1. **Model Context Protocol (MCP)**
2. **Agent-to-Agent (A2A) Protocol**
3. **Agentic Routing Patterns**
4. **State Management**
5. **Security & Governance**
6. **Observability cho Multi-Agent**
7. **Live Demo: Research System (Google ADK)**

Mục Tiêu

Sau buổi học này, bạn sẽ:

1. Thiết kế và chạy **MCP server** cho AI tools (studio + governance)
2. Implement **A2A communication** giữa multiple agents
3. Xây dựng **agentic routing layer** với semantic routing
4. Quản lý **state, security, observability** trong multi-agent systems

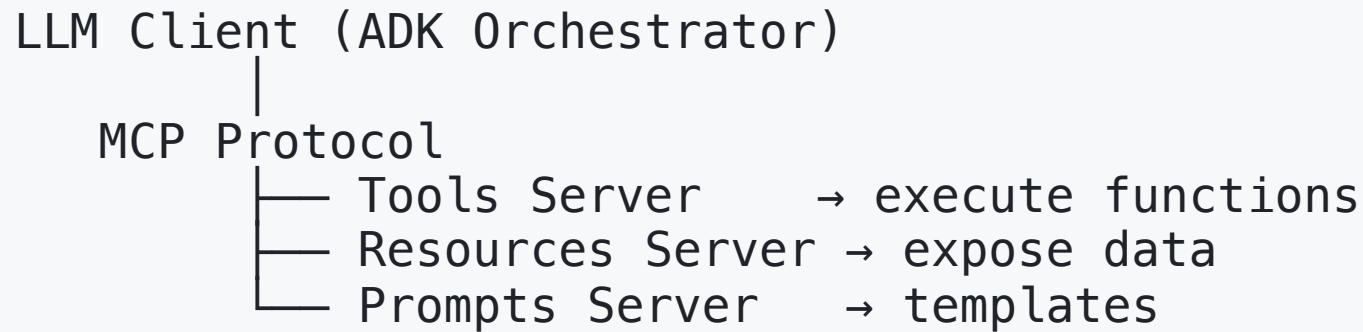
Luồng học: MCP → A2A → Routing → State → Security → Observability → Demo

Deliverable Cuối Ngày

MCP server + A2A message routing + multi-agent orchestration demo

- MCP server với research tools (stdio transport, governance guard)
- **Agent registry** với health check và capability metadata
- **Semantic router** gợi ý routing tới đúng specialist agent
- **Audit trace** (`governance_audit.jsonl`) cho full multi-agent flow
- **ADK Web UI** — orchestrator chat tại `localhost:8000`

MCP: Infrastructure Layer cho LLM Tools



- **MCP** = open protocol cho LLM ↔ tools/data (Anthropic standard, widely adopted)
- **Transport:** `stdio` (local/subprocess), HTTP+SSE (remote), WebSocket (streaming)
- **Build once, use everywhere** — MCP server works với Claude, GPT, ADK, LangChain

MCP Server Hosting & Registry

Local / Lab deployment

- **stdio**: ADK spawn MCP subprocess tự động
- **Remote (tùy chọn)**: FastAPI + uvicorn trên `localhost`
- Health check: agent card / tool ping
- **Stateless design** — scale bằng nhiều process/worker
- Script khởi động: `start_capstone.sh`

Lab: `lab_utils/agent_registry.py` + agent cards tại `/.well-known/agent-card.json`

Agent Registry pattern

- Agents đăng ký URL + capabilities
- Query registry → tìm available tools/agents
- Auto-register on startup, deregister on shutdown
- Capability metadata: input/output schemas
- Version-aware routing (khi cần)

MCP Server Implementation

```
from mcp.server import Server

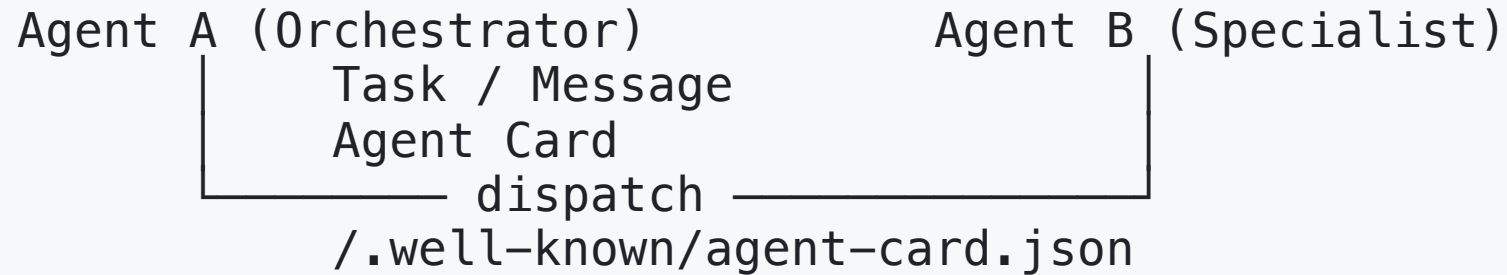
app = Server("research-tools")

@app.tool()
async def search(query: str) -> str:
    """Search documents."""
    return format_results(await db.search(query))

@app.tool()
async def sql_query(sql: str) -> str:
    """Execute read-only SQL."""
    return await db.execute(sql)
```

- Docstring rõ ràng — LLM đọc để quyết định dùng tool nào
- Type hints + validate inputs trước execute
- **Read-only by default** cho SQL tools

A2A: Microservices cho AI Agents



- **Google A2A protocol:** Agent Card + Task + Message
- **Agent Card:** JSON tại `/.well-known/agent-card.json` — capabilities, schemas
- **Auth:** OAuth 2.0 + JWT cho production; lab dùng localhost + policy allowlist
- **ADK:** `transfer_to_agent` + `RemoteA2aAgent` cho dispatch

A2A Task Lifecycle

Submitted → Working → Input Required → Completed
❏ Failed / Canceled

Key design: Task có thể pause ở "**Input Required**" — agent yêu cầu thêm thông tin từ caller trước khi tiếp tục.

Không phải mọi task đều straight-through.

Orchestrator decompose request → dispatch sub-tasks → aggregate kết quả

Routing Strategies So Sánh

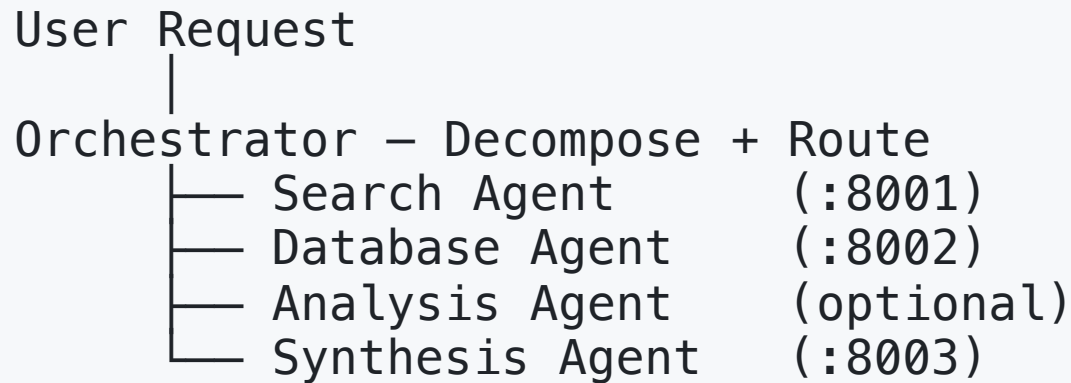
Strategy	Ưu điểm	Nhược điểm	Khi nào dùng
Keyword-based	Fast, simple	Brittle	≤5 agents
Embedding-based	Robust, scalable	Cần embeddings	5–50 agents
LLM-based	Flexible, context-aware	Expensive, slow	Complex routing

Semantic routing: embed request → cosine similarity với agent capability descriptions → top-k candidates.

Fallback chains: primary agent → fallback agent → human escalation — never dead-end users.

Lab: `lab_utils/semantic_router.py` + `suggest_routing()` tool

Orchestrator Pattern: Task Decomposition



- Orchestrator nhận intent → gọi `suggest_routing()` → `transfer_to_agent`
- MCP tools chỉ orchestrator được phép gọi (capability matrix)
- **Load balancing (production):** round-robin / least-busy giữa nhiều instance cùng loại agent

Lab capstone: 1 orchestrator + 3 A2A specialists

Stateless vs Stateful Agents

Stateless (khuyến nghị trước)

- Horizontal scaling dễ — thêm process/worker
- Externalize state → Redis / Postgres / file
- Bất kỳ instance xử lý bất kỳ request
- Restart/upgrade không mất context (state ngoài)

Stateful (khi cần)

- Long-running conversations
- Sticky sessions qua `session_id`
- In-memory state cho performance
- Cần drain connections khi scale down
- Trade-off: harder to scale

Best practice: Design stateless first, add state externally. Redis (<1hr), PostgreSQL (persistent history).

Distributed State & Event Sourcing

- **Redis:** short-lived context, TTL-based
- **PostgreSQL:** persistent conversation history
- **Distributed locks:** prevent race conditions khi nhiều agent cùng truy cập resource
- **Event bus (Kafka):** append-only, replay-able — shared memory cho agents
- **Context window management:** summarize older turns khi approaching token limit

Lab: ADK session state + auto `trace_id` trong governance callbacks

Security cho Agentic Systems

Control	Mô tả
Rate Limiting	Per-agent & per-user limits trên tool calls
Sandbox Execution	Agent code trong isolated environment
Human-in-the-Loop	High-stakes actions cần approval
Audit Log	Timestamp, agent ID, I/O mọi invocation
Minimal Capability	Agent chỉ có tools nó cần

Principle: Defense in depth — mỗi layer là independent protection.

Agent bypass một layer → vẫn bị rate limit + audit + HITL gate.

Governance: Quy Trình & Controls

Capability matrix (lab `policy.json`):

- **Research agent:** search, read — NO write
- **Database agent:** SELECT only — NO DDL
- **Email agent:** draft — human approves send
- **Code agent:** execute in sandbox only

HITL triggers: write production DB · send external email · arbitrary code · spend > budget

Runaway prevention: max 50 tool calls/task · max execution time · cost ceiling per request

Lab: `lab_utils/governance/` — *guard, audit, ADK callbacks*

Distributed Tracing: Single Trace ID

```
Trace ID: abc-123
├── Orchestrator Span (2.3s)
│   ├── Search Agent (0.8s)
│   │   └── web_search 0.5s
│   └── DB Agent (1.1s)
│       └── sql_query 0.9s
```

- **W3C Trace Context** qua A2A calls — single trace ID cho toàn bộ request
- **LangSmith / OpenTelemetry**: tree view orchestrator → sub-agents → tool calls
- **Deadlock detection**: alert nếu task stuck >30s without progress

Lab: auto `trace_id` + `logs/governance_audit.jsonl`

Agent Metrics & Cost Attribution

Operational metrics:

- `tasks_completed` / `tasks_failed`
- `avg_task_duration` · `tool_call_count`
- Error rate per agent type · queue depth

Cost attribution:

- Token cost per agent · cost per task type
- Multi-agent vs single-agent comparison
- Budget alerts per agent pool

Dashboard (production): Grafana panels — throughput, p95 latency, cost/request, error rate.

Lab: bảng `agent_metrics` + governance audit JSONL

Live Demo: Multi-Agent Research System

Kiến trúc (Google ADK):

```
:8000 ADK Web (orchestrator)
├── A2A → search_agent      :8001
├── A2A → database_agent    :8002
├── A2A → synthesis_agent   :8003
└── MCP   research_tools   (stdio)
```

1. Khởi động: `bash scripts/start_capstone.sh`
2. Agent cards + registry health check
3. Semantic routing qua `suggest_routing`
4. Governance: SQL guard, rate limit, HITL, audit log
5. Trace tree trong audit log / ADK Web events

Lab #26

Mục tiêu: Multi-Agent Infrastructure

Deliverable: Hệ 4 agent (orchestrator + 3 specialists) qua A2A + MCP + governance audit

Stack: Google ADK · Conda (pii-env) · Jupyter notebook · ADK Web UI

Thời gian: ~2 giờ

```
conda activate pii-env
export PYTHONPATH=$PWD
bash scripts/start_capstone.sh
# → http://localhost:8000
```

Tổng kết — Key Takeaways

1. **MCP standardizes tool interface** — build once, use across many LLM frameworks.
Deploy local với stdio hoặc uvicorn + registry pattern.
2. **Agent-to-Agent = microservices cho AI** — same principles: contracts, auth, observability. Google A2A + ADK `transfer_to_agent` .
3. **Routing intelligence tại orchestrator layer** > hardcoded agent selection — invest in semantic routing + fallback chains.
4. **Governance không phải optional** — capability matrix, audit, HITL là production requirement.

Tiếp theo & Bài tập

Ngày 27: Data Observability & Advanced Monitoring

Great Expectations advanced suites · Monte Carlo-style anomaly detection · dbt tests · SLO engineering

Bài tập:

- Hoàn thành **Lab 26**: Multi-Agent Infrastructure
- Đọc trước: Great Expectations checkpoint documentation
- Đọc trước: Google SRE Workbook — SLO engineering chapter

Hỏi & Đáp

Câu hỏi nào về MCP, A2A protocol, agentic routing, hay multi-agent observability?

Cảm ơn!

AICB-P2T2 · Ngày 26

MCP/A2A Infrastructure & Agentic Routing

lms.vinuni.edu.vn